operation itself, as you saw with the lock-free queue in chapter 7. In an extreme case, if a thread is waiting for a task to be completed and that task hasn't yet been started by any thread, the waiting thread might perform the task in entirety itself or another task that's incomplete. You saw an example of this in listing 8.1, where the sort function repeatedly tries to sort outstanding chunks as long as the chunks it needs are not yet sorted.

Rather than adding threads to ensure that all available processors are being used, sometimes it pays to add threads to ensure that external events are handled in a timely manner, to increase the *responsiveness* of the system.

8.4.4. Improving responsiveness with concurrency

Most modern graphical user interface frameworks are *event driven*; the user performs actions on the user interface by pressing keys or moving the mouse, which generate a series of events or messages that the application then handles. The system may also generate messages or events on its own. In order to ensure that all events and messages are correctly handled, the application typically has an event loop that looks like this:

```
while(true)
{
    event_data event=get_event();
    if(event.type==quit)
        break;
    process(event);
}
```

Obviously, the details of the API will vary, but the structure is generally the same: wait for an event, do whatever processing is necessary to handle it, and then wait for the next one. If you have a single-threaded application, this can make long-running tasks hard to write, as described in section 8.1.3. In order to ensure that user input is handled in a timely manner, `get_event()` and `process()` must be called with reasonable frequency, whatever the application is doing. This means that either the task must periodically suspend itself and return control to the event loop, or the `get_event()/process()` code must be called from within the code at convenient points. Either option complicates the implementation of the task.